Code review

General overview ◂

Detailed overview ◂

This part is a code review of the reference script, dlojz.py. All the optimizations of the course will be applied to this script, so everyone must know it well.

Two levels:

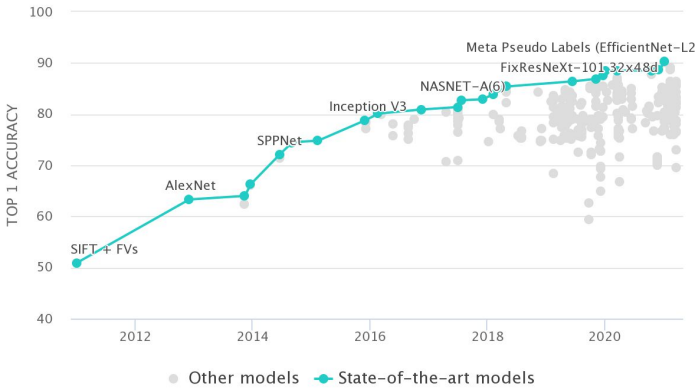
- General overview: the data, the model and the main blocks of a training loop.
- Detailed overview: the script itself, block by block.

Data – ImageNet

Goal:
Classification (1000 classes)

Dataset:
Train dataset: **1.2 M** labeled images
Validation dataset: **50,000** labeled images
https://www.image-net.org/





We stay on a simple and well-known task: image classification on ImageNet (ILSVRC 2012).

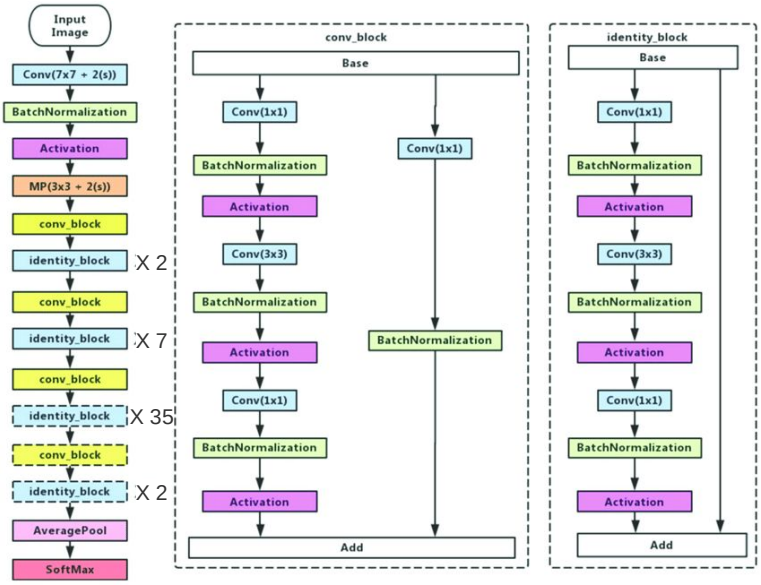
- 1000 classes.
- About 1.28 million training images and 50,000 validation images.
- Images have different sizes; they are resized to 224×224 in the DataLoader.

Why ImageNet? It is the reference benchmark of computer vision, and it is big enough to show real bottlenecks: data loading, GPU compute, memory, communication.

The chart shows the top-1 accuracy race on ImageNet: from AlexNet (2012, about 63%) to more than 90% today.

On Jean Zay the dataset is already available in the shared scratch ($ALL_CCFRSCRATCH/imagenet), no need to download it.

ImageNet – ResNet-152



ResNet-152

- Residual (skip) connections
- Bottleneck blocks
  - 1×1 → 3×3 → 1×1 convolutions
- 3 + 8 + 36 + 3 blocks
  - ≈ 60 M parameters, ≈ 11.5 GMACs per 224×224 image

The model is ResNet-152 (torchvision, trained from scratch).

- Residual connections: each block learns a correction added to its input (skip connection). This lets very deep networks train without vanishing gradients.

- Bottleneck block: 1×1 conv to reduce the channels, 3×3 conv, 1×1 conv to expand again. Each conv is followed by BatchNorm, so the convs have no bias.

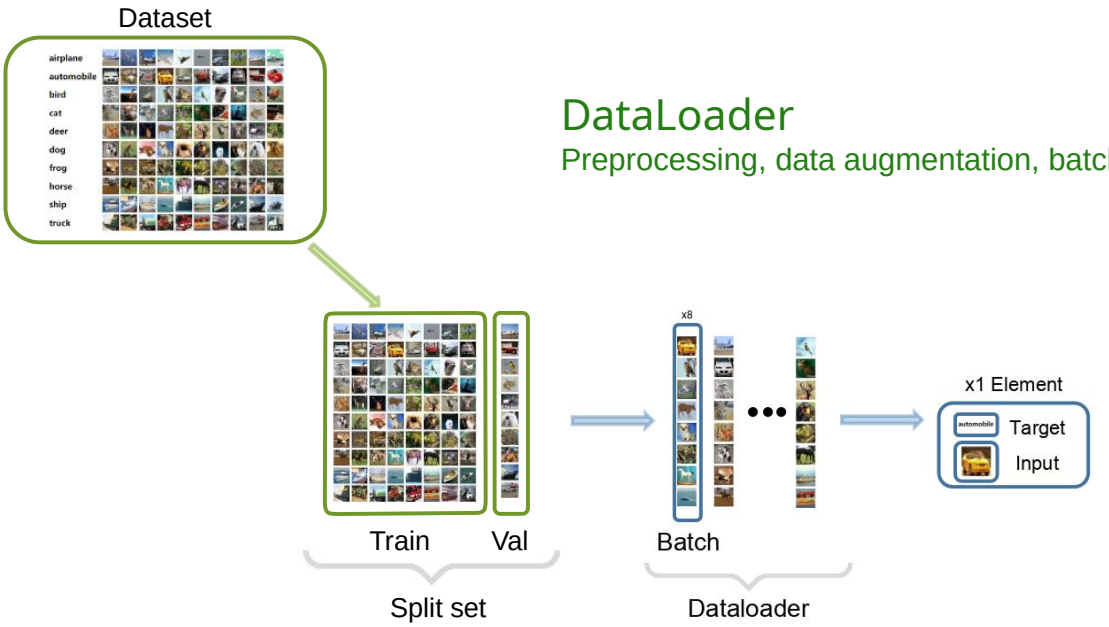
- Left of the figure: the stem (7×7 conv, max pooling), then 4 stages. Each stage = one conv_block (downsampling, projection shortcut) + identity_blocks. ResNet-152 has 3, 8, 36 and 3 blocks, so 2, 7, 35 and 2 identity blocks.

- Global average pooling at the end: the network accepts any input size.

Orders of magnitude: about 60 M parameters, about 11.5 GMACs (23 GFLOPs) per 224×224 image for the forward pass.

Same structure as ResNet-50; only the number of blocks in stage 3 and 4 changes.

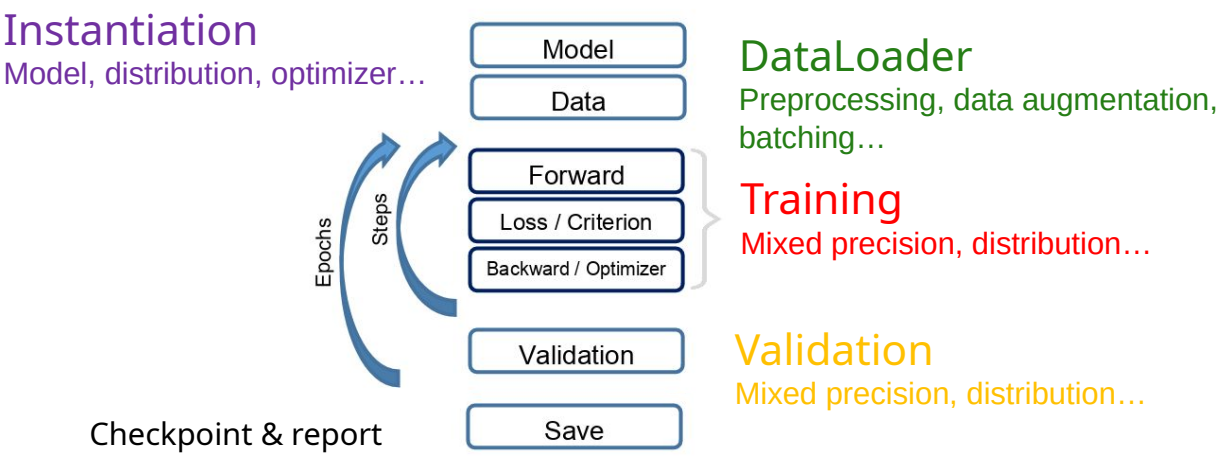
Training Loop – DataLoader



- The dataset is split into a training set and a validation set.
- The DataLoader reads the images, applies preprocessing and data augmentation, and groups them into batches.
- Each element of a batch is a pair (input image, target label).

The DataLoader runs on the CPU, in worker processes. If it is too slow, the GPU waits for data: this is the data-loading bottleneck we will measure later.

Training Loop

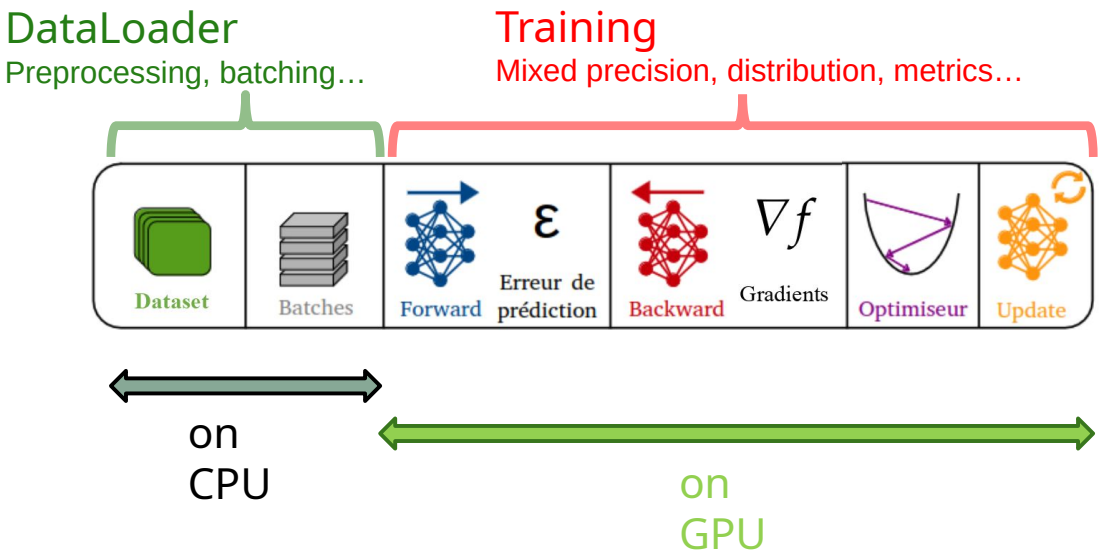


The full training loop, from top to bottom:

1. Instantiation: model, distribution, loss, optimizer, metrics.
2. DataLoader: provides the batches.
3. Training: for each batch, forward, loss, backward, optimizer step. This is repeated for each step and for each epoch.
4. Validation: forward only, no gradient, to measure the accuracy on unseen data.
5. Checkpoint & report: save the weights and print the timings.

Mixed precision and distribution appear in several blocks: we will see them in the next chapters.

Training step – GPU acceleration



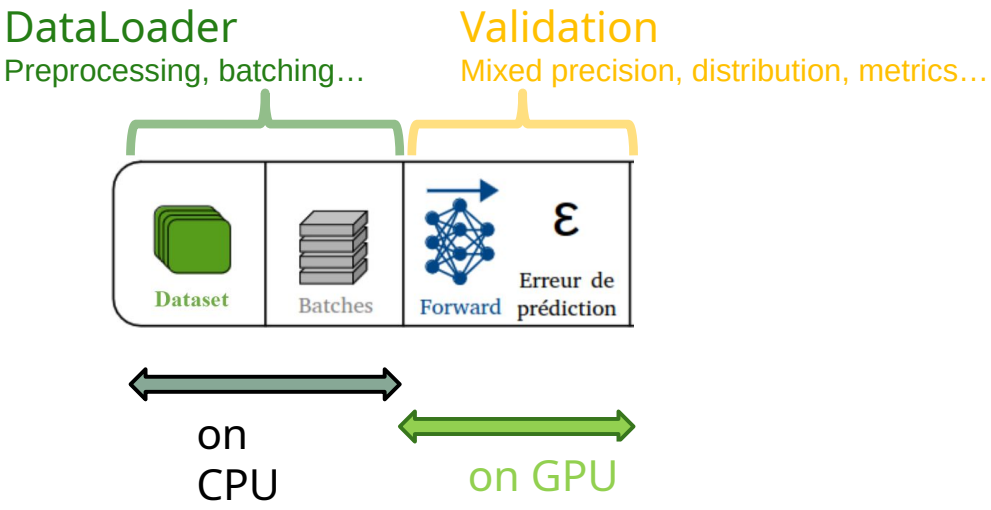
One training step.

- On the CPU: the DataLoader reads and prepares the batch.
- The batch is copied to the GPU.
- On the GPU: forward pass, loss (prediction error), backward pass (gradients), optimizer update.

All the heavy compute is on the GPU. The CPU must prepare the next batch while the GPU computes the current one, otherwise the GPU waits.

The red part is where we will apply most optimizations: mixed precision, channels last, distribution, etc.

Validation step – GPU acceleration



One validation step: same idea, but forward only.

- No backward and no optimizer step
- Gradients are disabled (torch.no_grad), so less memory is used.
- No data augmentation: only resize, center crop and normalize.

Validation gives the accuracy on images the model has never seen. It is the real quality measure of the training.

dlojz.py – Import & run



```python
import os
import contextlib
import argparse
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
from torch.utils.checkpoint import checkpoint_sequential
import torch
import numpy as np
import apex

import idr_torch
from dlojz_chrono import Chronometer
from torchmetrics.aggregation import MeanMetric
from torchmetrics.classification import MulticlassAccuracy

import random
random.seed(123)
np.random.seed(123)
torch.manual_seed(123)
```

reproducibility

Imports: os, argparse, PyTorch, torchvision…

PyTorch NumPy torchvision

Chronometer (DLO-JZ)

Time logging & in-house profiler



```python
if __name__ == '__main__':

    # display info
    if idr_torch.rank == 0:
        print(">>> Training on ", len(idr_torch.hostnames), " nodes and ", idr_torch.size, " processes")
    train()
```

idr_torch (Jean Zay)

Distribution helpers: rank, size, nodes

- Imports: standard Python libraries, PyTorch, torchvision, torchmetrics.
- Seeds are fixed (random, numpy, torch) for reproducibility.
- idr_torch: an IDRIS module that reads the Slurm environment and gives the rank, the number of processes and the node list. It makes distribution on Jean Zay easy.
- Chronometer: a small DLO-JZ class to measure time (per step, forward, backward, validation). It is our home-made profiler for the course.
- The main block prints the number of nodes and processes, then calls train().

dlojz.py – Argument parser



```python
## import ... ## Add here the libraries to import

VAL_BATCH_SIZE=256

#########################################################################################################
def train():                                                                                            #
    parser = argparse.ArgumentParser()                                                                  #
    parser.add_argument('-b', '--batch-size', default=128, type =int,                                   #
                        help='batch size per GPU')                                                      #
    parser.add_argument('-e','--epochs', default=1, type=int,                                           #
                        help='number of total epochs to run')                                           #
    parser.add_argument('--image-size', default=224, type=int,                                          #
                        help='Image size')                                                              #
    parser.add_argument('--lr', default=0.1, type=float,                                                #
                        help='learning rate')                                                           #
    parser.add_argument('--wd', default=0., type=float,                                                 #
                        help='weight decay')                                                            #
    parser.add_argument('--mom', default=0.9, type=float,                                               #
                        help='momentum')                                                                #
    parser.add_argument('--test', default=False, action='store_true',     ##    DON'T MODIFY    ######## #
                        help='Test 50 iterations')                                                      #
    parser.add_argument('--test-nsteps', default='50', type=int,                                        #
                        help='the number of steps in test mode')                                        #
    parser.add_argument('--num-workers', default=10, type=int,                                          #
                        help='num workers in dataloader')                                               #
    parser.add_argument('--persistent-workers', default=True, action=argparse.BooleanOptionalAction,   #
                        help='activate persistent workers in dataloader')                               #
    parser.add_argument('--pin-memory', default=True, action=argparse.BooleanOptionalAction,            #
                        help='activate pin memory option in dataloader')                                #
    parser.add_argument('--non-blocking', default=True, action=argparse.BooleanOptionalAction,          #
                        help='activate asynchronuous GPU transfer')                                     #
    parser.add_argument('--prefetch-factor', default=3, type=int,                                       #
                        help='prefectch factor in dataloader')                                          #
    parser.add_argument('--drop-last', default=False, action=argparse.BooleanOptionalAction,            #
                        help='activate drop_last option in dataloader')                                 #
#########################################################################################################

    ## Add parser arguments

    args = parser.parse_args()
```

Configurable arguments:

`--batch-size` : batch size per GPU
`--epochs` : number of epochs
`--image-size` : image size

Optimizer:
`--lr` : learning rate
`--wd` : weight decay
`--mom` : momentum

Test mode:
`--test` : test mode
`--test-nsteps` : number of steps in test mode

DataLoader optimization:
`--num-workers`
`--persistent-workers`
`--pin-memory`
`--non-blocking`
`--prefetch-factor`
`--drop-last`

- --batch-size: batch size per GPU (the global batch size is batch size × number of GPUs).

- --epochs, --image-size.

- Optimizer: learning rate, weight decay, momentum.

- Test mode: --test runs only a few steps (--test-nsteps, default 50) to measure performance quickly. We use it in almost all labs.

- DataLoader options: number of workers, persistent workers, pinned memory, non-blocking copy, prefetch factor, drop last.

dlojz.py – Instantiation



```python
## chronometer initialisation
chrono = Chronometer()

# define model
model = models.resnet152()
archi_model = 'Resnet-152'

if idr_torch.rank == 0: print(f'model: {archi_model}')
if idr_torch.rank == 0: print('number of parameters: {}'.format(sum([p.numel()
                                        for p in model.parameters()])))
# distribute batch size (mini-batch)
num_replica = idr_torch.size
mini_batch_size = args.batch_size
global_batch_size = mini_batch_size * num_replica

if idr_torch.rank == 0:
    print(f'global batch size: {global_batch_size} - mini batch size: {mini_batch_size}')

# define loss function (criterion) and optimizer
criterion = torch.nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.SGD(model.parameters(), args.lr, momentum=args.mom, weight_decay=args.wd)

if idr_torch.rank == 0: print(f'Optimizer: {optimizer}')

# define metrics
train_metric = {}
train_metric['loss'] = MeanMetric()
train_metric['acc'] = MulticlassAccuracy(num_classes=1000, average='micro')
val_metric = {}
val_metric['loss'] = MeanMetric()
val_metric['acc'] = MulticlassAccuracy(num_classes=1000, average='micro')
```

```python
#LR scheduler to accelerate the training time
scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=args.lr,
                                                steps_per_epoch=N_batch, epochs=args.epochs)
```



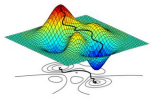
Chronometer



Model: ResNet-152

mini batch size ⟺ global batch size

CrossEntropyLoss



SGD optimizer



Metrics



LR scheduler

Instantiation: everything is created once, before the loop.

- Chronometer.
- Model: ResNet-152 from torchvision, moved to the GPU.
- Batch size: mini batch size = per GPU; global batch size = mini batch size × number of GPUs.
- Loss: CrossEntropyLoss (with label smoothing).
- Optimizer: SGD with momentum.
- LR scheduler: OneCycleLR, to reach a good accuracy in fewer epochs.

dlojz.py – DataLoader



```
#########  DATALOADER ############
# Define a transform to pre-process the training images.

if idr_torch.rank == 0: print(f"DATALOADER {args.num_workers} {args.persistent_workers} {args.pin_memory}

transform = transforms.Compose([
        transforms.RandomResizedCrop(args.image_size),  # Random resize - Data Augmentation
        transforms.RandomHorizontalFlip(),              # Horizontal Flip - Data Augmentation
        transforms.ToTensor(),                          # convert the PIL Image to a tensor
        transforms.Normalize(mean=(0.485, 0.456, 0.406),
                             std=(0.229, 0.224, 0.225))
        ])


train_dataset = torchvision.datasets.ImageNet(root=os.environ['ALL_CCFRSCRATCH']+'/imagenet',
                                              transform=transform)

train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
                                           batch_size=mini_batch_size,
                                           shuffle=True,
                                           num_workers=args.num_workers,
                                           persistent_workers=args.persistent_workers,
                                           pin_memory=args.pin_memory,
                                           prefetch_factor=args.prefetch_factor,
                                           drop_last=args.drop_last)


val_transform = transforms.Compose([
          transforms.Resize((256, 256)),
          transforms.CenterCrop(224),
          transforms.ToTensor(),   # convert the PIL Image to a tensor
          transforms.Normalize(mean=(0.485, 0.456, 0.406),
                               std=(0.229, 0.224, 0.225))])

val_dataset = torchvision.datasets.ImageNet(root=os.environ['ALL_CCFRSCRATCH']+'/imagenet', split='val',
                    transform=val_transform)

val_loader = torch.utils.data.DataLoader(dataset=val_dataset,
                                         batch_size=VAL_BATCH_SIZE,
                                         shuffle=False,
                                         num_workers=args.num_workers,
                                         persistent_workers=args.persistent_workers,
                                         pin_memory=args.pin_memory,
                                         prefetch_factor=args.prefetch_factor,
                                         drop_last=args.drop_last)


N_batch = len(train_loader)
N_val_batch = len(val_loader)
N_val = len(val_dataset)
```

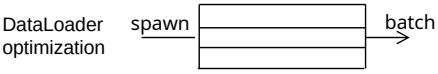
Train dataset:

RandomResizedCrop
RandomHorizontalFlip
+ Normalize



Shuffling

DataLoader optimization



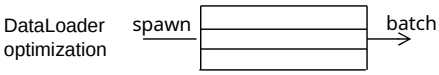
Validation dataset:

Resize
CenterCrop
+ Normalize



No shuffling

DataLoader optimization



Training:

- RandomResizedCrop + RandomHorizontalFlip: data augmentation, the model sees a different version of each image at each epoch.
- Normalize with the ImageNet mean and std.
- Shuffle = True.

Validation:

- Resize + CenterCrop + Normalize: deterministic, no augmentation.
- Shuffle = False.

Both use the same optimization options: worker processes are spawned once, then they produce batches in parallel.

dlojz.py – Training



```python
chrono.start()

#### TRAINING ############
for epoch in range(args.epochs):

    if args.test: chrono.next_iter()
    if idr_torch.rank == 0: chrono.tac_time(clear=True)

    for i, (images, labels) in enumerate(train_loader):

        csteps = i + 1 + epoch * N_batch
        if args.test and csteps > args.test_nsteps: break
        if i == 0 and idr_torch.rank == 0:
            print(f'image batch shape : {images.size()}')

        if args.test: chrono.forward()

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)

        if args.test: chrono.backward()

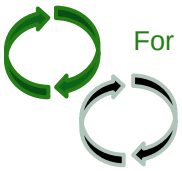
        loss.backward()
        optimizer.step()

        # Metric mesurement
        train_metric['loss'].update(loss)
        train_metric['acc'].update(outputs, labels)

        if args.test: chrono.update()

        if ((i + 1) % (N_batch//10) == 0 or i == N_batch - 1):
            train_loss, accuracy = train_metric['loss'].compute(), train_metric['acc'].compute()
            train_metric['loss'].reset(), train_metric['acc'].reset()
            if idr_torch.rank == 0:
                print('Epoch [{}/{}], Step [{}/{}], Time: {:.3f}, Loss: {:.4f}, Acc:{:.4f}'.format(
                      epoch + 1, args.epochs, i+1, N_batch,
                      chrono.tac_time(), train_loss, accuracy))

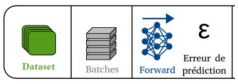

        # scheduler update
        scheduler.step()
```



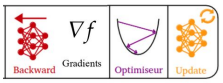
For each epoch

For each *batch*
test mode: 50 steps

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
```




```
loss.backward()
optimizer.step()
```





Accumulate metrics (loss, accuracy)
Compute and print them 10× per epoch



LR scheduler step (every batch)

The training loop.

- For each epoch, for each batch:
- copy images and labels to the GPU;
- zero_grad, forward, loss;
- backward, optimizer step.
- Metrics are accumulated at each step, then computed and printed 10 times per epoch.
- The LR scheduler (OneCycleLR) is stepped after every batch.
- In test mode, the loop stops after 50 steps.

The chrono calls (forward, backward, update) are only active in test mode, to measure each phase.

dlojz.py – Validation



```
#### VALIDATION #############
if ((i == N_batch - 1) or (args.test and i==args.test_nsteps-1)) :

    chrono.validation()
    model.eval()

    for iv, (val_images, val_labels) in enumerate(val_loader):

        # Runs the forward pass with no grad mode.
        with torch.no_grad():
            val_outputs = model(val_images)
            val_loss = criterion(val_outputs, val_labels)

        val_metric['loss'].update(val_loss)
        val_metric['acc'].update(val_outputs, val_labels)

        if args.test and iv >= 20: break

    val_loss, val_accuracy = val_metric['loss'].compute(), val_metric['acc'].compute()
    val_metric['loss'].reset(), val_metric['acc'].reset()

    model.train()
    chrono.validation()
    if not args.test and idr_torch.rank == 0:
        print('##EVALUATION STEP##')
        print('Epoch [{}/{}], Validation Loss: {:.4f}, Validation Accuracy: {:.4f}'.format(
                                epoch + 1, args.epochs, val_loss, val_accuracy))
        print(">>> Validation complete in: " + str(chrono.val_time))

#### END OF VALIDATION #############
```

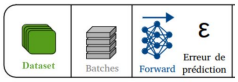


After each epoch
(or at the end of test mode)



For each *validation batch*
(test mode: 20 steps)

```
# Runs the forward pass with no grad mode.
with torch.no_grad():
    val_outputs = model(val_images)
    loss = criterion(val_outputs, val_labels)
```





Accumulate metrics (loss, accuracy)

- model.eval(): BatchNorm uses its running statistics.
- torch.no_grad(): no gradient, less memory, faster.
- For each validation batch: forward and loss, then metrics are accumulated.
- In test mode, only about 20 validation batches are used.
- At the end: loss and accuracy are printed, and model.train() switches back to training mode.

dlojz.py – Checkpoint & Report



```
chrono.stop()
if idr_torch.rank == 0:
    chrono.display()
    print(">>> Number of batch per epoch: {}".format(N_batch))
    print(f'Max Memory Allocated {torch.cuda.max_memory_allocated()} Bytes')


# Save last checkpoint
if not args.test and idr_torch.rank == 0:
    checkpoint_path = f"checkpoints/{os.environ['SLURM_JOBID']}_{global_batch_size}.pt"
    torch.save(model.state_dict(), checkpoint_path)
    print("Last epoch checkpointed to " + checkpoint_path)
```

Report: timings + max GPU memory

Checkpoint saved at the end of training (not in test mode)



Trained model weights

- The Chronometer prints its report: total time, time per step, throughput (images per second), validation time.
- We also print the maximum GPU memory allocated during the run. Throughput and peak memory are the two numbers we will follow all along the course.
- Checkpoint: the model weights (state_dict) are saved at the end of training, only for a real run, not in test mode.

TP0: Set up your environment



- Open a terminal and clone the repository

```
local:~$ ssh jean-zay

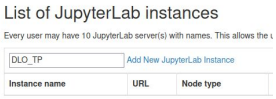
jz:~$ cd $WORK
jz:~$ git clone https://github.com/idriscnrs/dlojz_tp/
```



- Open Firefox
- Go to jupyterhub.idris.fr

TP0: JupyterHub

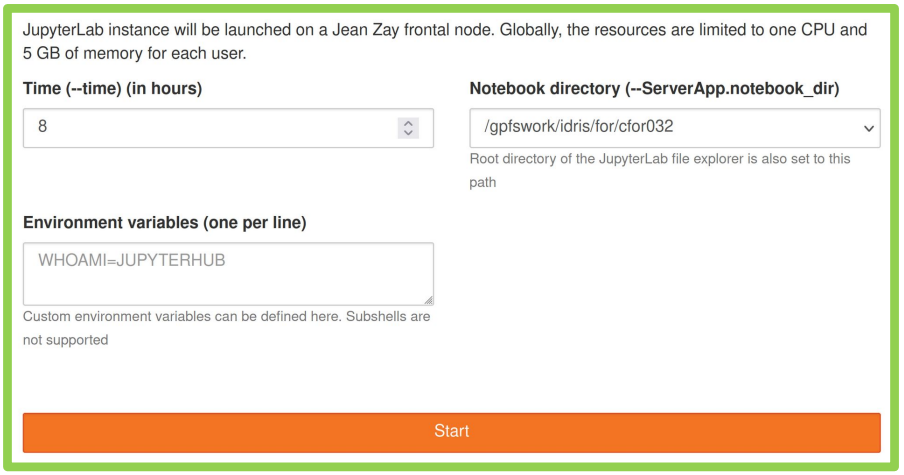
- Log in with your training account
- Add a new instance



- Select the spawner ‘Interactive’



- Fill in the configuration



- Start

TP0: Notebook



- Open the notebook DLO-JZ_Jour1.ipynb
- Select the kernel pytorch-gpu/py3/2.8.0 (top right) if it is not detected
- Choose a pseudonym
- Submit a first job
- Explore the reference script and its options